REVIEW 3: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell

Project Title: Smart Archive Agent

Team Details: Team 12

Team Members: Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 5: Testing, Evaluation, and Results

5.1 Test Cases & Scenarios

The Smart Archive Agent was tested in three complementary ways: (1) an automated pytest suite of **58 tests** that runs every tool against real files with the real Linux utilities, (2) a labelled set of natural-language requests used to measure intent accuracy (Section 5.2), and (3) a live rehearsal of the complete demonstration in an interactive terminal (Chapter 7). All results in this chapter were produced on Ubuntu 26.04.1 LTS on WSL 2, Python 3.14.4, GNU tar 1.35, XZ Utils 5.8.3, GnuPG 2.4.8, 16 logical CPUs. The scenarios below follow the three categories of the review template.

Test file	Tests	What it verifies
tests/test_intent.py	7	Intent recognition and mapping of paths to tool arguments.
tests/test_end_to_end.py	8	Path validation, tar.xz create/list/inspect/extract round trip, bit-exact xz compression and decompression, overwrite protection, checksums.
tests/test_agent.py	37	Chained plans, compression levels, guardrails, protected folders, plan validation, cancellation, tool schemas, invalid model tool calls, zip round trip, GPG encrypt/decrypt with right and wrong passphrase, Zip Slip.
tests/test_integration.py	6	Integration contract for the Agentic Shell: manifest and agent.json, prefixed tool names, JSON-string arguments, command line, runnable integration example.
Total	58	58 passed in 1.51 s (python3 -m pytest -q).

5.1.1 Simple Intents

A simple intent maps to one tool call. The template's example ("Find all PDFs created yesterday") belongs to the Semantic File Search agent; the equivalent single-step requests for this agent are archive, compress, inspect, extract, encrypt and decrypt operations. The table lists representative requests, the tool and parameters the agent selected, and the outcome that was verified.

Request	Tool selected	Verified outcome
archive ./docs quickly	create_archive (gz, level fast = gzip -1)	docs.tar.gz created next to the source (live demo, Fig. 7.3).
create a zip file of ./src	create_archive (zip)	Valid src.zip; extracts back to identical files (test_zip_round_trip).
compress ./README.md quickly	compress_file (gz, fast)	README.md.gz created; original file untouched.
inspect archive ./docs.tar.gz	inspect_archive	Type and 16 entries reported; nothing written to disk.
what is inside ./demo.zip	list_archive	Entry names listed without extraction.
extract ./backup.tar.xz into ./out	extract_archive	Every member checked, then files restored.
decompress ./data.csv.xz	decompress_file	Original bytes restored exactly (bit-exact test).
Encrypt ./README.md	encrypt_file	README.md.gpg created with GPG symmetric AES-256.

5.1.2 Complex Chained Intents

A chained intent needs several tools in a fixed order, where each step consumes the output of the previous one. The planner builds the whole chain first, shows it with absolute paths, and runs nothing until the user confirms. The first row is the exact prompt from the Agent #12 brief.

Request	Plan produced	Verified outcome
Zip up my project folder, compress it heavily, and encrypt it.	1. create_archive (xz, max = xz -9e) 2. encrypt_file (AES-256) 3. remove_intermediate (plaintext archive)	Only project.tar.xz.gpg remains; decrypting it restores the original files (test_brief_example_end_to_end_and_decrypt; live demo).
decrypt ../project.tar.xz.gpg and extract it	1. decrypt_file 2. extract_archive 3. remove_intermediate	Files restored to project-extracted/; the temporary decrypted archive is deleted.
Archive ./src to ../src.tar.xz and encrypt ../src.tar.xz ../src.tar.xz.gpg	1. create_archive 2. encrypt_file	The user named the intermediate archive, so it is kept.
squeeze ./src as small as possible, then lock it with a password	1. create_archive (max) 2. encrypt_file 3. remove_intermediate	Same chain from different wording.
back up ./project quickly and encrypt it	1. create_archive (gz, fast) 2. encrypt_file	"quickly" selects gzip -1 instead of xz.

	3. remove_intermediate	

If any step fails, the chain stops, already-finished steps are reported (completed_steps), and a failing step leaves no partial output. A wrong passphrase, for example, removes the half-written file and reports "wrong passphrase".

5.1.3 Malicious/Destructive Inputs

Destructive requests are stopped by four independent layers: a regular-expression blacklist applied to the raw text before any planning, the scope check, schema validation of every tool call, and path validation inside each tool (protected system folders, no overwrites, Zip Slip). No request ever reaches a shell, because utilities are started with subprocess.run() and an argument list.

Input	Layer that stopped it	Result
Delete everything on my drive	Guardrail blacklist (mass deletion)	Blocked before planning.
rm -rf /	Guardrail blacklist (rm -rf)	Blocked before planning.
archive ./src; rm -rf ~	Guardrail blacklist (rm -rf, ; injection)	Blocked; the archive part is not run either.
compress $(cat /etc/shadow)	Guardrail blacklist (command substitution)	Blocked.
dd if=/dev/zero of=/dev/sda	Guardrail blacklist (raw disk write)	Blocked.
archive / to ./root.tar.xz	Guardrail blacklist (whole root filesystem)	Blocked.
extract ./sample.tar.xz into /usr/bin	Protected-location check	Refused before anything is written.
Archive containing ../../escaped.txt	Zip Slip check in extract_archive	Refused; destination folder not even created.
Model tool call bash("rm -rf /")	Schema validation in execute_tool	Rejected as an unknown tool; nothing runs.
What is the weather today?	Scope check	Reported as outside this agent's scope.

5.2 Performance Metrics

All metrics were measured with benchmarks/evaluate.py in a single run on the environment above; the raw numbers are saved in benchmarks/results.json and can be reproduced with python3 benchmarks/evaluate.py.

5.2.1 Intent Translation Accuracy

A request counts as correct only if the agent produces exactly the labelled sequence of tools (and the labelled compression level, where one is given), or blocks / rejects it when that is the expected outcome. Accuracy was measured on separate sets so that tuning does not inflate the result.

Request set	Requests	Correct	Accuracy
Development set (used while building; tuned on)	65	65	100.0%
First held-out set, before the vocabulary rewrite	20	10	50.0%
Second held-out set (written afterwards, never tuned on)	20	16	80.0%

Held-out category	Correct / total	Accuracy
Simple intents	6/10	60.0%
Chained intents	5/5	100.0%
Malicious inputs	3/3	100.0%
Out-of-scope requests	2/2	100.0%

Analysis. The honest estimate of accuracy on new wording is **80%**. The first held-out set scored only 50%: the original keyword list missed everyday verbs ("unlock", "show me the files") and matched "zip" inside "gzip". The vocabulary was rewritten with whole-word matching and a fresh held-out set was written. Its four misses ("turn ./project into a tar.gz", "get the files out of …", "what files does … contain", "check what type of archive … is") were all safe failures: three were rejected as out of scope and one proposed an extra archive that the user would see before confirming. Chained and malicious requests were handled correctly in every set. The weak point is unusual single-verb wording, which is exactly where the Agentic Shell's LLM, calling our tools through their JSON schemas, is expected to help.

5.2.2 Latency (Time-to-Execution)

Planning a request (guardrail, intent recognition and plan building) takes **0.066 ms** on average (95th percentile 0.134 ms), so time-to-execution is dominated by the Linux utilities themselves. Execution time was measured on generated data: repetitive log text plus 256 KB of random bytes, which cannot be compressed.

Input	Format and level	Time	Output size (% of input)
1.25 MB	tar.gz, fast (gzip -1)	10 ms	20.59%
1.25 MB	tar.gz, normal (gzip -6)	13 ms	20.31%
1.25 MB	tar.bz2 (bzip2 -9)	203 ms	20.2%
1.25 MB	tar.xz, normal (xz -6)	75 ms	20.12%
1.25 MB	tar.xz, max (xz -9e)	137 ms	20.12%

1.25 MB	zip (deflate 6)	8 ms	20.3%
10.25 MB	tar.gz, fast (gzip -1)	17 ms	3.06%
10.25 MB	tar.gz, normal (gzip -6)	27 ms	2.77%
10.25 MB	tar.bz2 (bzip2 -9)	1666 ms	2.49%
10.25 MB	tar.xz, normal (xz -6)	206 ms	2.47%
10.25 MB	tar.xz, max (xz -9e)	366 ms	2.47%
10.25 MB	zip (deflate 6)	40 ms	2.77%

End-to-end operation	Time
Brief's example: plan + tar.xz (xz -9e) + GPG AES-256 + clean-up, 1.25 MB folder	0.29 s
GPG AES-256 encryption of a 0.253 MB archive	0.13 s (0.13–0.82 s across runs, depending on whether gpg-agent was already running)

Observations. gzip is the fastest and zip is comparable. bzip2 was the slowest by far (about 1.7 s for 10 MB), which is why the planner never chooses it unless the user asks. xz -9e took about 1.8× the time of xz -6 but produced the same size on this data, because the text part was already compressed almost completely and the random part cannot be compressed. Heavier levels pay off only on data with long-range redundancy.

5.2.3 Error Recovery Rate

In this design the LLM never writes a shell command; it can only propose a call to one of the eight tools. A "syntactically invalid command" therefore appears as an invalid or unsafe tool call. The benchmark sent 18 such calls to execute_tool(). A case counts as recovered only if the agent returned a structured error, raised no exception, created no file and left existing files unchanged.

Invalid or unsafe call	Agent response
Unknown tool name	unknown tool 'delete_files'
Hallucinated shell tool	unknown tool 'bash'
Missing required argument	missing argument 'output_path'
Extra injected argument	unexpected argument 'shell'
Invalid enum value	argument 'level' must be one of ['fast', 'normal', 'max']
Wrong argument type	argument 'archive_path' must be a non-empty string
Empty path	argument 'source_path' must be a non-empty string

Arguments not an object	arguments must be an object
Non-existent source	Path does not exist: <path>
Overwrite existing output	Output already exists: <path>
Write into /etc	Refusing to write into a protected system location: /etc/backup.tar.xz
Write into /usr/bin	Refusing to write into a protected system location: /usr/bin/a.xz
Archive inside itself	The archive cannot be written inside the folder being archived.
Folder passed to compress_file	Compression input must be a file. Folders are archived with create_archive.
Zip Slip archive	Archive contains a path that escapes the destination.
Unsupported decompress format	Cannot infer compression format. Use a .gz, .bz2, or .xz file.
Same input and output	The output path must differ from the source path.
Wrong decryption passphrase	GPG decryption failed. Check the passphrase. (wrong passphrase)

Result: 18/18 = 100.0% recovered. The interactive shell never terminated on an error during testing; every failure is returned as a dictionary with success = false and an error message, and the session continues. The benchmark also caught a real defect: the first version created the extraction folder before scanning a malicious archive, leaving an empty folder behind; the scan now runs before anything is created.

Chapter 6: Conclusion & Future Scope

6.1 Summary of Contributions

The project delivers a working prototype that bridges conversational text and real system execution for the archive domain of the Agentic Shell:

- **Working prototype:** the brief's request "Zip up my project folder, compress it heavily, and encrypt it" runs end to end: tar.xz at xz -9e, GPG AES-256, and removal of the unencrypted copy.
- **LLM-ready tools:** eight tools (create, extract, list, inspect, compress, decompress, encrypt, decrypt) with JSON Schema definitions available in generic, OpenAI and Anthropic formats through get_tools(), and one safe entry point, execute_tool(), for the base shell to call.
- **Offline planner:** a deterministic planner that needs no API key or network, chains steps, maps words such as "heavily" and "quickly" to compression levels, chooses safe default file names, and cleans up intermediate plaintext files.
- **Layered safety:** destructive-command blacklist, scope check, schema validation, human confirmation, protected system folders, Zip Slip prevention, no overwrites, and passphrases passed to gpg on standard input.

- **Ready for integration:** installable with one pip command from GitHub, standard library only, a unique package name (smart_archive_agent) and prefixed tool names so it can be loaded next to the other 34 agents, a machine-readable manifest (agent.json) for the shell's router, an integration guide with a runnable example, and a --doctor check for the required Linux tools.
- **Measured quality:** 58 automated tests, 80% intent accuracy on unseen phrasings, sub-millisecond planning, and 100.0% recovery from invalid tool calls.

6.2 Challenges Encountered

- **Managing state and context history across multiple command executions:** within one request the agent passes each step's output path to the next step and records which files it created, so the clean-up step can delete only those files and a failed step can report exactly how far the chain got. Across separate requests the agent keeps no conversation history, so a follow-up such as "now encrypt it" must name the file. This keeps behaviour predictable and safe, but it limits natural multi-turn conversation.
- **Generalising beyond fixed keywords:** the first held-out test scored 50%, and "zip" was matched inside "gzip". Rewriting the vocabulary with whole-word matching raised unseen-phrasing accuracy to 80%, but rule-based recognition still cannot cover every wording.
- **GPG inside a subprocess:** the first version failed with "Inappropriate ioctl for device" because gpg tried to open its own passphrase prompt. It now uses loopback pinentry, reads the passphrase from standard input, and disables passphrase caching so a wrong passphrase is always detected.
- **Ordering of safety checks:** Zip Slip detection originally ran after the destination folder was created; the error-recovery benchmark exposed this and the check now runs first.
- **Running on other machines:** the agent must work on the teacher's machine, not only ours. The package was renamed from the generic name app, the code uses only the standard library, and --doctor lists missing tools with the exact apt command. Only Python 3.14 was available for testing; older versions are not yet verified.

6.3 Future Work

- **Multi-modal support:** voice-controlled requests through local speech-to-text (for example Whisper or Vosk) feeding the same planner and confirmation step.
- **Deeper local file indexing:** an index of file types and modification times so requests such as "archive all spreadsheets from last quarter" can select their sources, possibly reusing the Semantic File Search agent.
- **Conversation context:** remembering the last output so that follow-ups like "now encrypt it" work, while still showing every plan before it runs.
- **MCP server:** exposing the same tool schemas through a Model Context Protocol server so any MCP client can use the agent.
- **Adaptive compression:** measuring file entropy first and skipping compression for data that is already compressed (JPEG, MP4, PDF), as the latency results suggest.
- **Wider platform testing:** running the test suite on Python 3.8–3.13 and on older Ubuntu releases used in the lab.

Chapter 7: Screenshots

The screenshots below were captured on Ubuntu (WSL 2). A small demo folder is used so the archive is quick to create. Each figure is preceded by the commands typed.

```
$ mkdir -p ~/demo/project && cp -r ~/SMART-ARCHIVE-AGENT/docs ~/demo/project/
$ cd ~/demo/project && export PYTHONPATH=~/SMART-ARCHIVE-AGENT
```

7.1 Environment Check

```
$ python3 -m smart_archive_agent --doctor
```

[Paste screenshot here]

Figure 7.1: Dependency check: Python and tar, gzip, bzip2, xz, gpg and file found.

7.2 Brief's Example: Plan, Confirmation and Encryption

```
$ python3 -m smart_archive_agent
You: Zip up my project folder, compress it heavily, and encrypt it.
Proceed? [y/N]: y
```

[Paste screenshot here]

Figure 7.2: Three-step plan shown before execution; archive at xz -9e, GPG AES-256, plaintext copy removed.

7.3 Fast Compression and Inspection

```
You: archive ./docs quickly
You: inspect archive ./docs.tar.gz
```

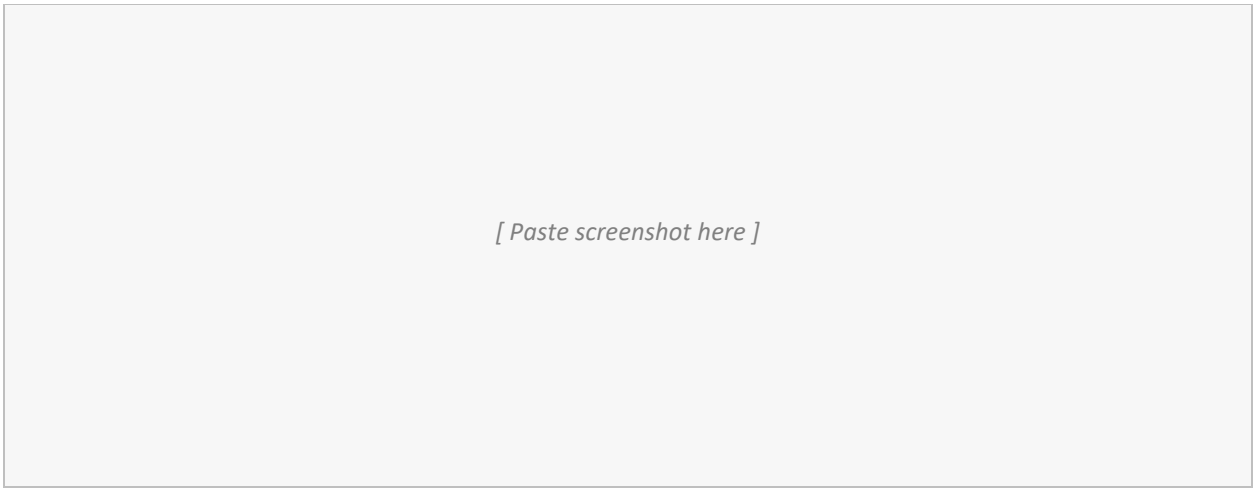


Figure 7.3: "quickly" selects gzip -1; inspection lists the archive without extracting it.

7.4 Decrypting and Extracting in One Request

```
You: decrypt ../project.tar.xz.gpg and extract it
```

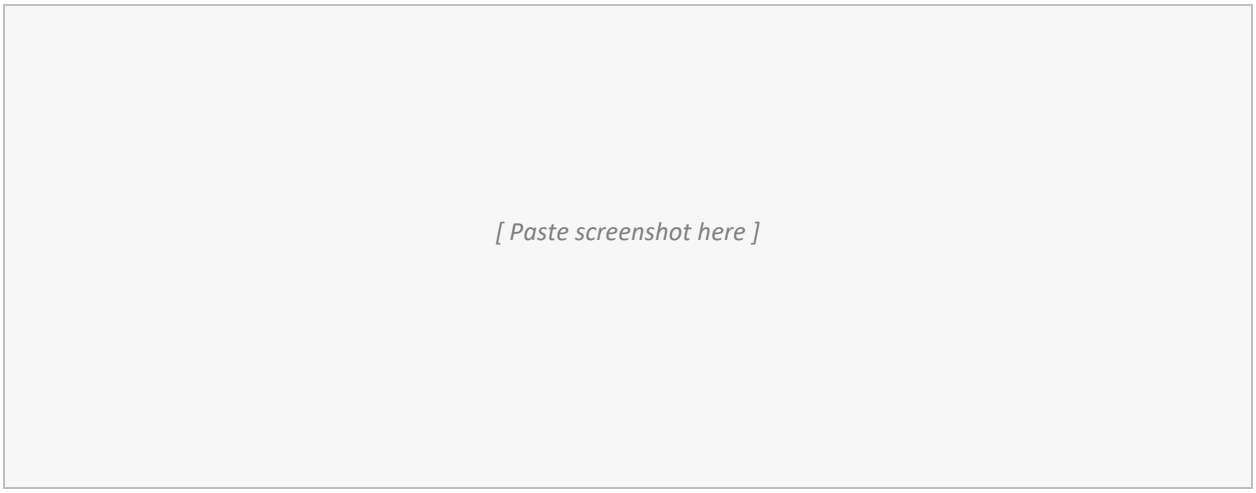


Figure 7.4: Decrypt, extract and clean-up chained from one sentence.

7.5 Guardrails

```
You: rm -rf /
You: archive ./docs; rm -rf ~
You: What is the weather today?
You: exit
```

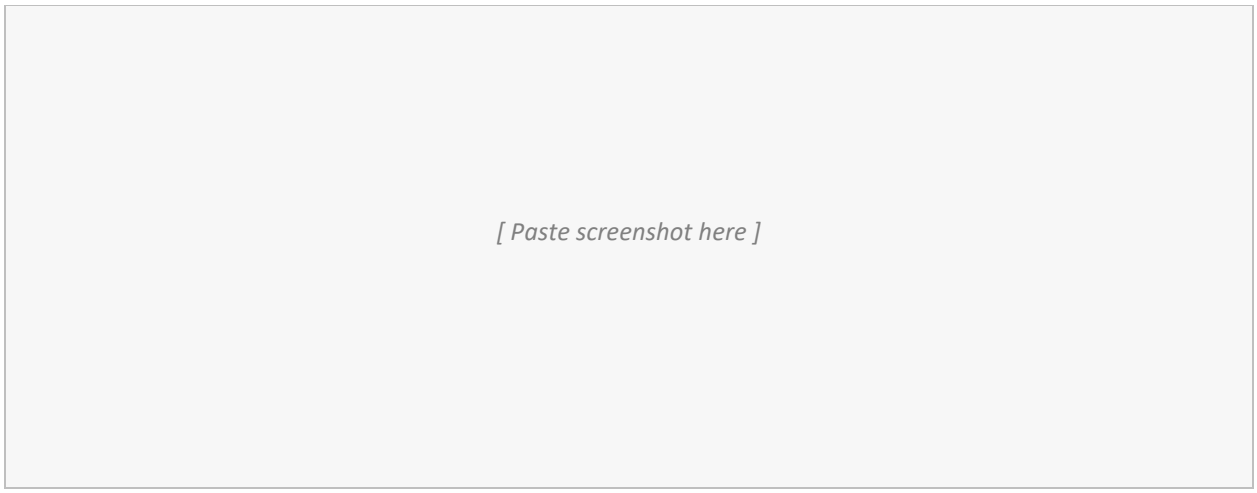


Figure 7.5: Destructive requests blocked before planning; an unrelated request reported as out of scope.

7.6 Automated Test Suite

```
$ cd ~/SMART-ARCHIVE-AGENT && python3 -m pytest -q
```

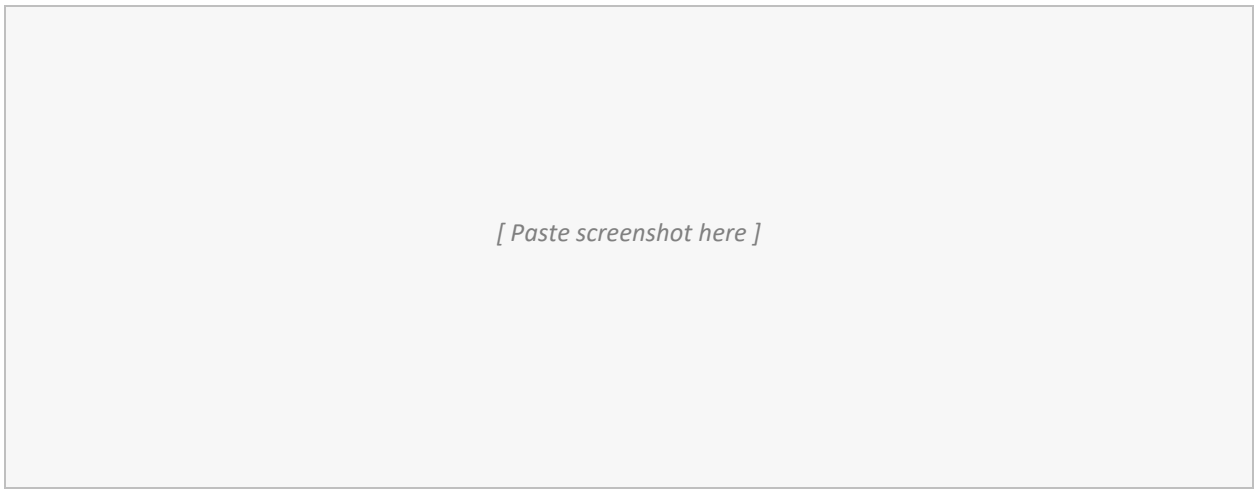


Figure 7.6: 58 tests passed.

7.7 Benchmark Results

```
$ python3 benchmarks/evaluate.py
```



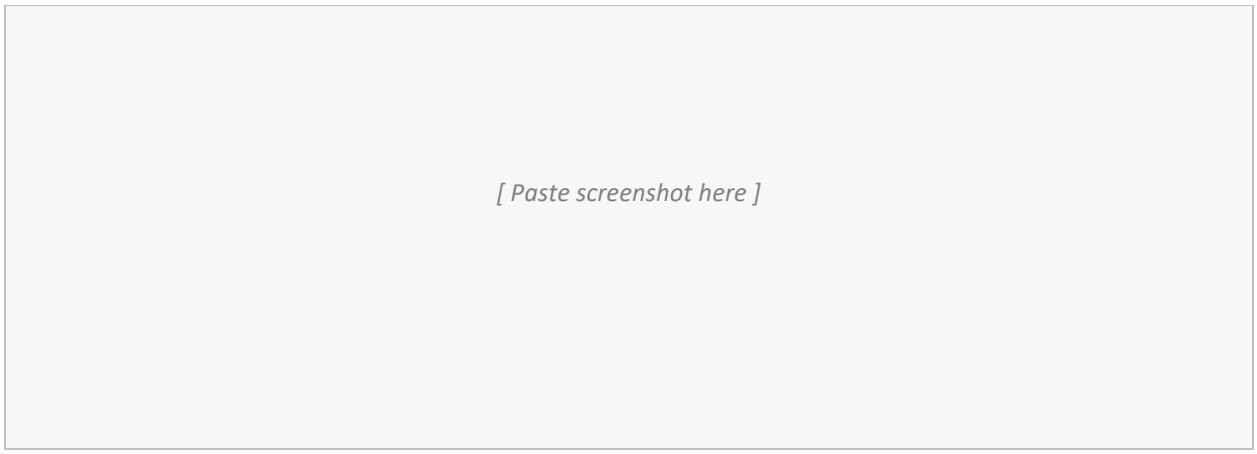


Figure 7.7: Intent accuracy, latency and error-recovery measurements used in Section 5.2.

References

[1] Free Software Foundation, "GNU tar: an archiver tool," manual for GNU tar 1.35. Available: https://www.gnu.org/software/tar/manual/

[2] Tukaani Project, "XZ Utils," documentation. Available: https://tukaani.org/xz/

[3] W. Koch et al., "The GNU Privacy Guard (GnuPG) Documentation." Available: https://gnupg.org/documentation/

[4] Python Software Foundation, "subprocess — Subprocess management," Python 3 documentation. Available: https://docs.python.org/3/library/subprocess.html

[5] Python Software Foundation, "tarfile — Extraction filters" and PEP 706, "Filter for tarfile.extractall." Available: https://peps.python.org/pep-0706/

[6] Snyk Security Research, "Zip Slip Vulnerability," 2018. Available: https://security.snyk.io/research/zip-slip-vulnerability

[7] OWASP Foundation, "Path Traversal." Available: https://owasp.org/www-community/attacks/Path_Traversal

[8] Anthropic, "Model Context Protocol specification." Available: https://modelcontextprotocol.io/